

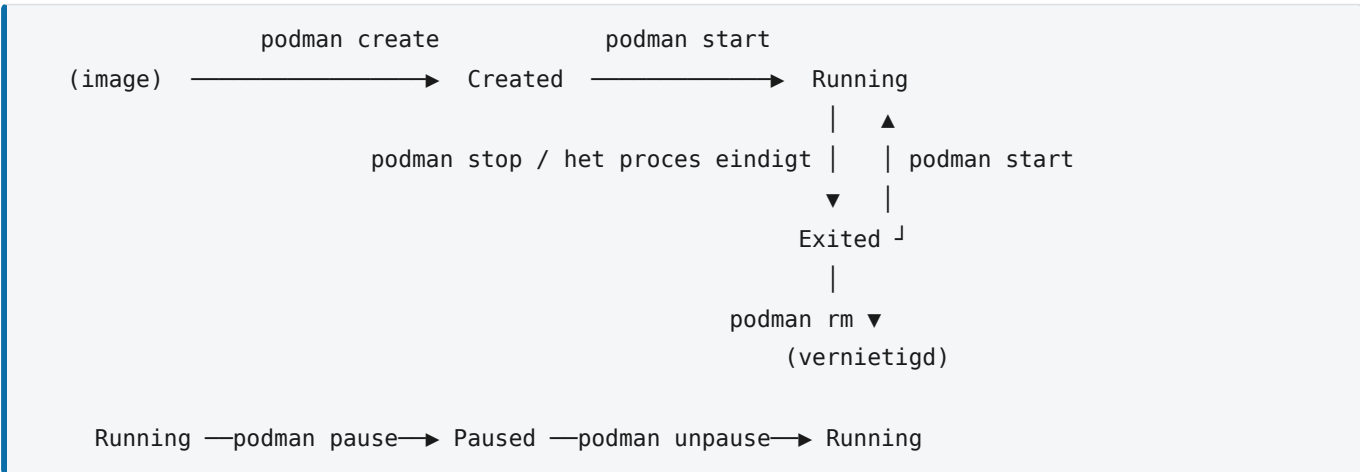
Lab 03 — De levenscyclus van een container

Theorie — hoe een container geboren wordt, leeft, signalen krijgt en sterft; waarom PID 1 alles bepaalt, en wie je containers herstart als er geen daemon is.

Doelstellingen

- De toestanden van een container kennen, en de commando's die hem van de ene naar de andere brengen.
- Begrijpen waarom een container "vanzelf stopt".
- De relatie beheersen tussen het **hoofdproces**, de **signalen** en de **exitcode**.
- Kiezen tussen `exec` en `attach`, tussen voorgrond en achtergrond.
- Weten wat een *restart policy* echt doet — en wat ze zonder daemon niet kan.

1. De toestanden



Commando	Effect
<code>podman create</code>	Maakt de container klaar (schrijflaag, config) zonder hem te starten
<code>podman start</code>	Start het hoofdproces
<code>podman run</code>	<code>create</code> + <code>start</code> (+ <code>pull</code> als de image ontbreekt)
<code>podman stop</code>	Vraagt beleefd om te stoppen, forceert na een wachttijd
<code>podman kill</code>	Forceert de stop onmiddellijk
<code>podman restart</code>	<code>stop</code> gevolgd door <code>start</code>
<code>podman pause</code>	Bevriest de processen (cgroup <i>freezer</i>) zonder ze te stoppen
<code>podman rm</code>	Vernietigt de container inclusief zijn schrijflaag

Een `Exited`-container is niet dood: zijn configuratie, schrijflaag en logs blijven bestaan. Je kunt hem inspecteren, opnieuw starten of er bestanden uit halen. Alleen `rm` vernietigt echt.

2. De basisregel: een container leeft zolang zijn PID 1

ONTHOUDEN

Een container stopt op precies één moment: wanneer zijn **hoofdproces** eindigt. Niet vroeger, niet later. Meer is het niet.

Daarmee verklaar je bijna elke "mijn container stopt vanzelf":

- `podman run alpine` stopt meteen — het standaardcommando is `/bin/sh`, en zonder terminal heeft die shell niets te lezen en sluit hij af.
- `podman run nginx` blijft draaien — nginx werkt op de voorgrond en eindigt nooit uit zichzelf.
- Een script dat een dienst naar de **achtergrond** stuurt (`java -jar ... &`) sterft meteen: het script loopt af, en daarmee ook PID 1.

Daaruit volgt een ontwerpregel: **een image start haar dienst op de voorgrond**. Geen daemon, geen `systemd`, geen `nohup` in een container — de engine zelf speelt voor dienstbeheerder.

→ LINUX

Op een gewone Linux-machine is PID 1 `init` (tegenwoordig `systemd`): het eerste proces dat de kernel start, de voorouder van alle andere. De kernel behandelt het speciaal. Sterft PID 1, dan valt het hele systeem stil. En het **negeert standaard elk signaal** waarvoor het zelf geen handler installeerde — zo kan een onhandige `kill` de machine niet neerhalen. In een container erft *jouw applicatie* die `init`-status, met alle privileges en valkuilen die erbij horen.

Gevolg: een container is gemaakt voor **één hoofdproces**. Wie de API en de database in dezelfde container stopt, breekt het model — je kunt ze niet meer apart herstarten, bewaken of schalen.

3. Voorgrond, achtergrond, en het duo `-it`

```
podman run nginx                # voorgrond: de terminal is geblokkeerd, logs getoond
podman run -d nginx              # losgekoppeld: je krijgt je prompt terug, plus de container-ID
podman run -it alpine sh         # interactief: je krijgt een bruikbare shell
podman run --rm alpine date      # eenmalige uitvoering, container verwijderd bij afsluiten
```

`-it` wordt vaak verkeerd begrepen, want het zijn **twee aparte opties**. `-i` houdt de standaardinvoer open; zonder die optie ziet een shell een gesloten `stdin` en stopt hij meteen. `-t` wijst een pseudo-terminal toe: de prompt, de echo van je toetsen, `Ctrl+C`. In scripts en CI gebruik je alleen `-i`; als er een mens aan het toetsenbord zit, `-it`.

VALKUIL

Op de voorgrond stuurt `Ctrl+C` een `SIGINT` naar het proces in de container: nginx stopt. In `-it`-modus bereik je met `Ctrl+P` gevolgd door `Ctrl+Q` net het omgekeerde: je koppelt los **zonder de container te stoppen**.

PODMAN

Bij Docker betekent "losgekoppeld" dat de daemon de container bewaakt. Podman heeft geen daemon. Wanneer `podman run -d` je prompt teruggeeft, blijft `common` achter — een paar honderd KB, één per container. Het houdt de `stdout` / `stderr`-pijpen open, schrijft de logs weg en noteert de exitcode wanneer PID 1 sterft. Sluit je je WSL-sessie, dan sterven `common` en je containers mee... tenzij `systemd` ze in leven houdt (deel 6).

4. Signalen: hoe een container sterft

`podman stop` is geen aan-uitknop. Er zit een protocol achter:

1. Podman stuurt **SIGTERM** naar PID 1: "sluit netjes af".
2. Het wacht een **respijtperiode** af, standaard 10 seconden (aanpasbaar met `-t`).
3. Leeft het proces dan nog, dan volgt **SIGKILL** — niet te onderscheppen, onmiddellijk. Podman meldt het: `StopSignal SIGTERM failed to stop container ... in 10 seconds, resorting to SIGKILL`.

`podman kill` springt rechtstreeks naar stap 3. En `podman rm -f` voert een volledige `stop` uit, de 10 seconden inbegrepen — vandaar de `-t 0` uit lab 01.

→ LINUX

Een **signaal** is een asynchrone melding van de kernel aan een proces. **SIGTERM** (15) vraagt om te stoppen en kan onderschept worden; **SIGKILL** (9) doodt onherroepelijk; **SIGINT** (2) is je `Ctrl+C`. Een programma "behandelt" een signaal door een *handler* te installeren; anders geldt de standaardactie — voor **SIGTERM** is dat sterven. PID 1 is de uitzondering: het heeft geen standaardactie en negeert het signaal gewoon.

In die 10 seconden werkt een goed geschreven Spring Boot-applicatie de lopende requests af, sluit ze de PostgreSQL-pool en meldt ze zich af bij de service discovery. Bij **SIGKILL** gebeurt daar niets van: requests worden afgebroken, verbindingen blijven aan databasekant hangen, en je gegevens zijn mogelijk inconsistent.

→ JAVA / SPRING BOOT

De JVM zet **SIGTERM** om in het uitvoeren van de **shutdown hooks** (`Runtime.addShutdownHook`). Spring Boot registreert er een die de context sluit: `@PreDestroy`, JDBC-pool, webserver. Met `server.shutdown=graceful` aanvaardt de server geen nieuwe verbindingen meer en laat hij de lopende requests afwerken. Dat alles werkt **alleen als SIGTERM de JVM bereikt**.

Twee valkuilen die het signaal onderweg tegenhouden:

1. Een shell tussen Podman en je applicatie. Dat gebeurt bij een opstartscript dat de applicatie zonder `exec` start, of bij de *shell*-vorm van `CMD` (`CMD java -jar app.jar` wordt `/bin/sh -c "java -jar app.jar"`). PID 1 is dan `sh`, en `sh` geeft **SIGTERM niet door** aan zijn kind. Java krijgt het signaal nooit, er verstrijken 10 seconden, en dan valt de doodsteek. De oplossing heeft twee kanten: gebruik de *exec*-vorm (`CMD ["java", "-jar", "app.jar"]`) en zet in een script `exec java -jar app.jar` op de laatste regel. Lab 04 diept dit syntaxisdetail uit; het bepaalt hoe netjes je containers stoppen.

2. De bijzondere status van PID 1. Een proces zonder **SIGTERM**-handler dat als PID 1 draait, reageert niet op `podman stop` en wordt na de wachttijd gedood. PID 1 moet bovendien weesprocessen "adopter"; anders stapelen de *zombies* zich op. Daarvoor bestaat `--init`: het zet een degelijke mini-init (`podman-init`) vóór je applicatie.

5. Exitcodes

```
podman run --rm alpine sh -c 'exit 3'; echo $?      # 3
podman ps -a --format 'table {{.Names}}\t{{.Status}}'
```

De exitcode van een container is die van zijn PID 1; hij blijft in de status staan (`Exited (3)`). Deze codes moet je herkennen:

Code	Gebruikelijke betekenis
0	Normale beëindiging
1	Generieke applicatiefout
125	De engine zelf faalde (ongeldige optie)
126	Commando gevonden maar niet uitvoerbaar (of <code>pasta</code> kon de poort niet openen)
127	Commando niet gevonden in de image
137	Gedood door <code>SIGKILL</code> (128+9) — <code>podman kill</code> , verstreken respijtperiode, of de OOM killer
143	Beëindigd door <code>SIGTERM</code> (128+15) — een nette <code>stop</code>

137 kom je in productie het vaakst tegen: een `stop` die de respijtperiode overschreed, of een overschreden geheugenlimiet. `podman inspect` geeft uitsluitsel: bij een geheugenprobleem staat `.State.OOMKilled` op `true`.

6. De *restart policies* — en wie ze toepast

```
podman run -d --restart=unless-stopped --name api mijn-api:1.0
```

Beleid	Gedrag
<code>no</code> (standaard)	Geen automatische herstart
<code>on-failure[:N]</code>	Herstart bij een exitcode $\neq 0$, hoogstens N keer
<code>always</code>	Herstart altijd, ook na een reboot van de host... als er iemand is om het te doen
<code>unless-stopped</code>	Zoals <code>always</code> , behalve als jij hem zelf gestopt hebt

Bij Docker past de daemon die regels toe, ook wanneer de machine opstart. Bij Podman herstart `common` de container zolang je sessie leeft — maar na een reboot is er niemand meer om het beleid uit te voeren.

PODMAN

Het antwoord van Podman heet **systemd**, de dienstbeheerder van Linux, via **Quadlet**: een bestand `~/.config/containers/systemd/api.container` van tien regels (`[Container]` , `Image=` , `PublishPort= ...`) plus `systemctl --user start api` . De container wordt een gewone dienst: hij start bij het booten, herstart bij falen en logt naar `journalctl` . Precies daarom wilde Podman geen eigen daemon — Linux heeft er al een (lab 10).

VALKUIL

`always` herstart een container zelfs na een handmatige `stop` , zodra de engine opnieuw start. `unless-stopped` onthoudt wat je bedoelde: op één machine is dat bijna altijd de juiste keuze.

7. Observeren en ingrijpen

```
podman logs -f --tail 50 api      # uitvoerstroom van PID 1
podman exec -it api sh            # nieuw proces IN de container
podman attach api                 # opnieuw aankoppelen op de bestaande PID 1
podman top api                    # processen van de container, gezien vanaf de host
podman stats api                  # CPU/geheugenverbruik in realtime
podman inspect api                # volledige toestand, JSON
podman cp api:/app/log.txt .      # een bestand uithalen, zelfs uit een gestopte container
podman events --since 10m         # het logboek van aanmaken, stoppen, sterven
```

`exec` **start een nieuw proces** in de namespaces van de container — dat is wat je wilt als je binnen wilt kijken. `attach` koppelt je aan de in- en uitvoer van de **bestaande PID 1**: een `Ctrl+C` stopt daar de hele container.

`podman logs` toont alleen wat PID 1 naar `stdout` / `stderr` schreef en wat `common` opving. Logt je applicatie naar een bestand, dan zie je hier niets. Vandaar de regel: **log naar de standaarduitvoer**. Spring Boot doet dat standaard; stel dus geen `logging.file.name` in.

8. In het bedrijf

- De Spring Boot-backend draait met `-d`, met `--restart=unless-stopped` onder Docker of als Quadlet-dienst onder Podman — of zonder beleid onder een orchestrator. Netjes afsluiten is een productiethema: `SIGTERM` dat aankomt + Spring *graceful shutdown* = uitrollen zonder één verloren request.
- Een diagnose volgt altijd dezelfde volgorde: `ps -a` (status, code), `logs`, `inspect` (OOM? configuratie?), en pas dan `exec` als de container nog leeft.

Onthouden

- Een container leeft precies zolang zijn hoofdproces (PID 1). Diensten draaien **op de voorgrond**: geen daemon, `&` of `systemd` in de container.
- `-i` houdt `stdin` open, `-t` wijst een terminal toe. `stop` = `SIGTERM`, respijtpriode, dan `SIGKILL` (Podman waarschuwt); `kill` = meteen `SIGKILL`; `rm -f` = volledige `stop` zonder `-t 0`. De `exec`-vorm (`["java", "-jar", "x.jar"]`) is onmisbaar om signalen te ontvangen.
- `137` = gedood (KILL of OOM), `143` = netjes gestopt (TERM), `127` = commando niet gevonden.
- `rm` vernietigt de gegevens van de container; `stop` niet. `exec` start een proces, `attach` koppelt aan PID 1. Zonder daemon loopt herstart bij het booten via `systemd` (Quadlet).

Woordenschat

PID 1: hoofdproces van de container. — **respijtpriode**: tijd tussen `SIGTERM` en `SIGKILL`. — **graceful shutdown**: nette stop die het lopende werk afrondt. — **restart policy**: regel voor automatische herstart. — **OOM killer**: de kernel doodt een proces als het geheugen op is. — **zombie**: beëindigd proces waarvan niemand de exitcode las. — **common**: toezichhouder van een Podman-container. — **Quadlet**: integratie van Podman in `systemd` (`.container` -bestanden).